



Lab 04 – Worksheet

Name: Zenon Leonard	ID: 11521	Section: T2
---------------------	-----------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes.



```
ASM task1a.s X ASM task1b.s X ASM task2.s X ASM task3.s X ASM ta
ASM task1a.s
1  main:
2      addi x10, x0, 6
3      jal  x1, fact
4      j    exit
5
6  fact:
7      # Save return address and current number
8      addi sp, sp, -8
9      sw   x1, 4(sp)
10     sw   x10, 0(sp)
11
12     addi x5, x10, -1
13     bge  x5, x0, L1
14
15     # Base case of ours
16     addi x10, x0, 1
17     addi sp, sp, 8
18     jalr x0, 0(x1)
19
20  L1:
21     # Recursive call
22     addi x10, x10, -1
23     jal  x1, fact
24
25     addi x6, x10, 0
26     lw   x10, 0(sp)
27     lw   x1, 4(sp)
28     addi sp, sp, 8
29
30     # Multiply n by fact(n-1)
31     mul  x10, x10, x6
32     jalr x0, 0(x1)
33
34  exit:
35  j    exit
```



**Add snip of relevant Registers / Memory locations where you think results are obtained. Make sure the irrelevant area of snip is cropped. Make sure the irrelevant area of snip is cropped.*

```
x07 (t2) = 0
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 720
x11 (a1) = 0
x12 (a2) = 0
x13 (a3) = 0
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 720
x11 (a1) = 0
x12 (a2) = 0
x13 (a3) = 0
```



Task 2

Provide appropriately commented codes

```
task2.s
1  main:
2      addi x10, x0, 6
3      addi x11, x0, 1
4      jal  x1, ntri
5      j    exit
6
7  ntri:
8      # Save return address and current value
9      addi sp, sp, -8
10     sw   x1, 4(sp)
11     sw   x10, 0(sp)
12
13     addi x5, x10, -1
14     bge  x5, x11, L1
15
16     # Base case: return 1
17     addi x10, x0, 1
18     addi sp, sp, 8
19     jalr x0, 0(x1)
20
21 L1:
22     # Recursive call: ntri(n-1)
23     addi x10, x10, -1
24     jal  x1, ntri
25
26     addi x6, x10, 0
27     lw   x10, 0(sp)
28     lw   x1, 4(sp)
29     addi sp, sp, 8
30
31     # Add current value to recursive result
32     add  x10, x10, x6
33     jalr x0, 0(x1)
34
35 exit:
36     j    exit
```



Add screenshot of your results

```
x07 (t2) = 0
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 21
x11 (a1) = 1
x12 (a2) = 0
x13 (a3) = 0
x14 (a4) = 0
```



Task 3

Provide appropriately commented codes



```
task1a.s X task1b.s X task2.s X task3.s X task4.s X
task3.s
1  .data
2  array:
3      .word 7, 3, 5, 1, 6, 2, 4
4  .text
5  .globl main
6
7  main:
8      la x10, array
9      addi x11, x0, 7
10     jal x1, bubble
11     j exit
12
13 bubble:
14     # Initialize outer loop
15     beq x11, x0, ret_bubble
16     addi x5, x0, 0
17
18 outer:
19     bge x5, x11, end_outer
20     addi x6, x0, 0
21
22 inner:
23     # Compare adjacent elements
24     addi x31, x11, -1
25     bge x6, x31, end_inner
26
27     slli x7, x6, 2
28     add x7, x10, x7
29     lw x28, 0(x7)
30
31     addi x29, x6, 1
32     slli x29, x29, 2
33     add x29, x10, x29
34     lw x30, 0(x29)
35
36     bge x28, x30, no_swap
37
38     # Swap elements if they are in the wrong order
39     sw x30, 0(x7)
40     sw x28, 0(x29)
41
42 no_swap:
43     addi x6, x6, 1
44     j inner
45
46 end_inner:
47     addi x5, x5, 1
48     j outer
49
50 end_outer:
51 ret_bubble:
52     jalr x0, 0(x1)
53
54 exit:
55     j exit
```



Add screenshots of your results

Memory viewer tool showing memory addresses and values. The tool has a dark theme and includes controls for jumping to an address, changing display format, and bytes per row.

Address	+0	+1	+2	+3
0x10000030	00	00	00	00
0x1000002C	00	00	00	00
0x10000028	00	00	00	00
0x10000024	00	00	00	00
0x10000020	00	00	00	00
0x1000001C	00	00	00	00
0x10000018	01	00	00	00
0x10000014	02	00	00	00
0x10000010	03	00	00	00
0x1000000C	04	00	00	00
0x10000008	05	00	00	00
0x10000004	06	00	00	00
0x10000000	07	00	00	00

Address: Up Down

Jump to: -- choose --

Display Format: Hex

Bytes per Row: 4



Task 4

Provide appropriately commented codes

```
task4.s
1  .text
2  .globl main
3  main:
4      li    x10, 10          #using number 10 for our series so we shall get 55 as result
5      jal   x1, fib
6
7      li    x17, 1
8      ecall
9
10     li    x17, 10          # we are exiting here
11     ecall
12
13     fib:
14         addi sp, sp, -12    #push
15         sw   x1, 8(sp)      # saving return address
16         sw   x10, 4(sp)
17
18         addi x5, x10, -2
19         bge  x5, x0, L1
20
21
22         addi sp, sp, 12
23         jalr x0, 0(x1)
24
25     L1:
26         addi x10, x10, -1
27         jal   x1, fib        # fib(n - 1)
28         sw   x10, 0(sp)
29
30         lw   x10, 4(sp)
31         addi x10, x10, -2
32         jal   x1, fib
33
34         lw   x6, 0(sp)
35         add  x10, x10, x6
36
37         lw   x1, 8(sp)
38         addi sp, sp, 12      #pop
39         jalr x0, 0(x1)
```

I HAVE SHOWN RECURSIVE FIBONACCI SERIES HENCE 10 SHALL GIVE AN OUTPUT OF 55

Add screenshots of your results



```
x07 (t2) = 0
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 55
x11 (a1) = 0
x12 (a2) = 0
x13 (a3) = 0
```



Lab 04 – Nest Procedures and Sorting

Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	AR 7 Report Submission & Github
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code/Simulation Model/Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary



			submitted after due date.	report is submitted timely.	figures, plots and tables.
--	--	--	---------------------------	-----------------------------	----------------------------